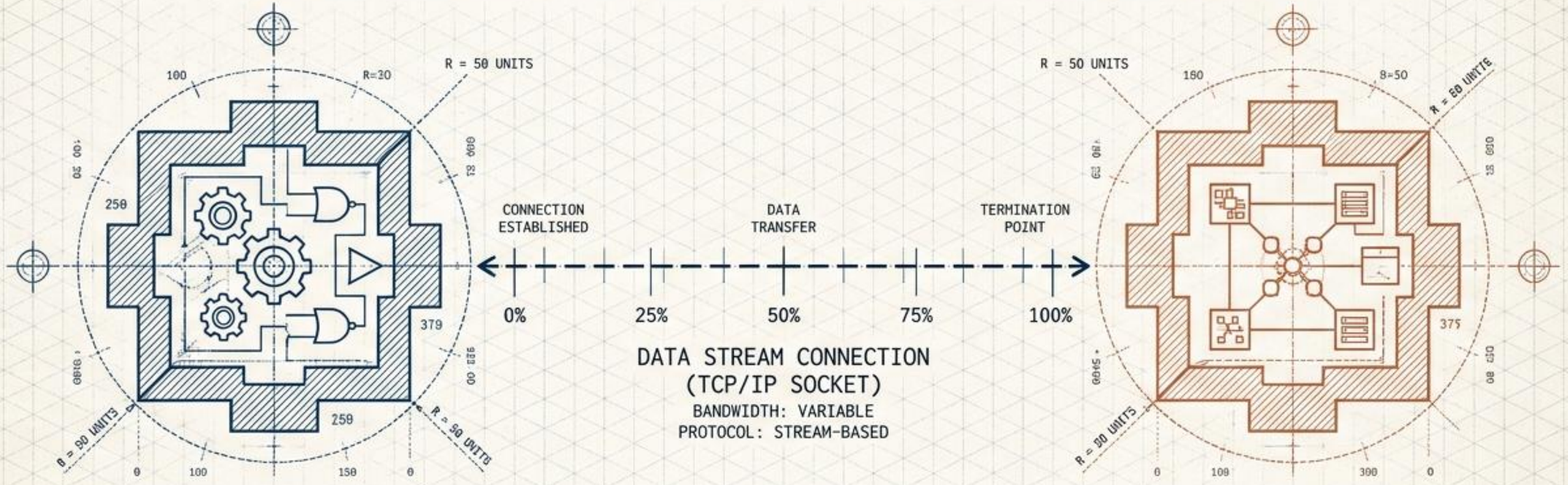
DWG NO. JNA-TITLE-001

JAVA NETWORK ARCHITECTURE

REV A

A VISUAL GUIDE TO SOCKETS, STREAMS, AND CONNECTION LIFECYCLES

DESIGNED FOR DEVELOPERS TRANSITIONING FROM LOCAL APPLICATIONS TO DISTRIBUTED SYSTEMS.

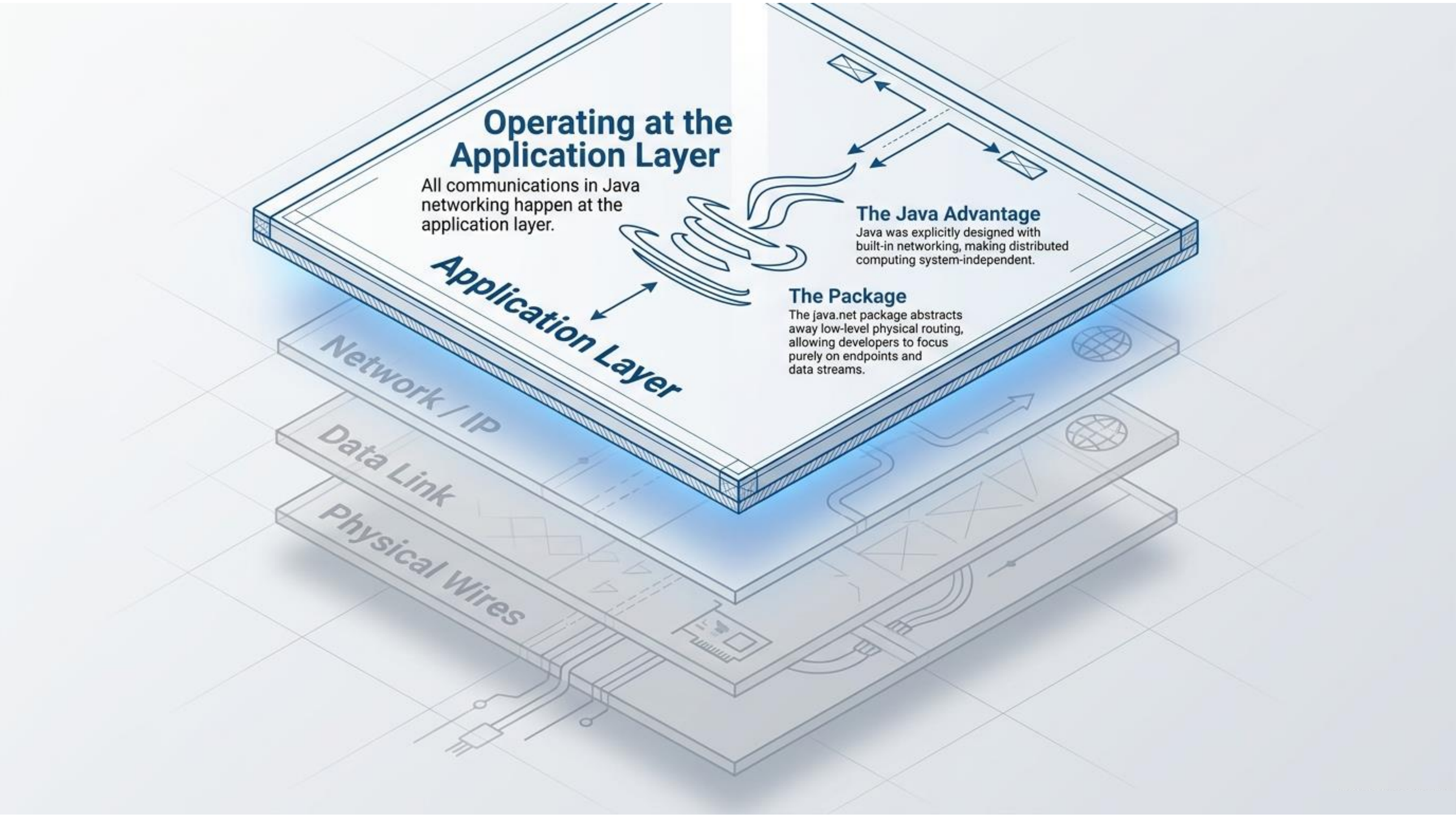


DATA STREAM CONNECTION
(TCP/IP SOCKET)
BANDWIDTH: VARIABLE
PROTOCOL: STREAM-BASED

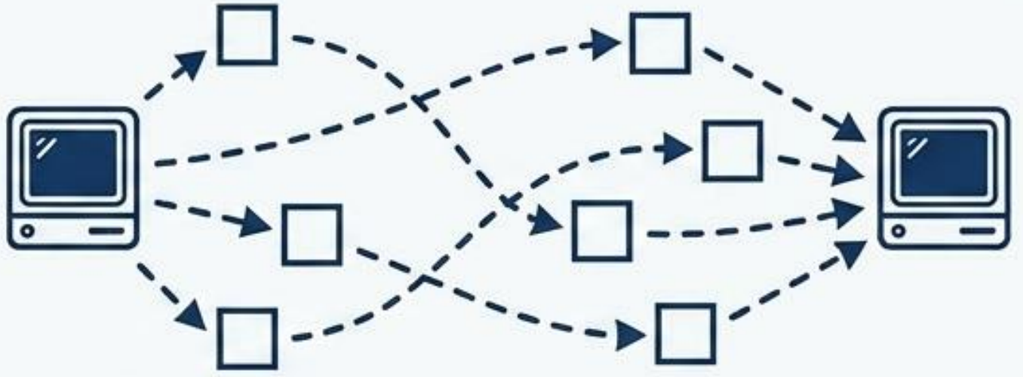
NODE A: LOCAL APPLICATION
X: 100, Y: 250, Z: 0

NODE B: DISTRIBUTED SYSTEM
X: 900, Y: 250, Z: 0

ENGINEERING SCHEMATIC - JAVA NETWORKING CORE


Operating at the Application Layer
All communications in Java networking happen at the application layer.
Application Layer
The Java Advantage
Java was explicitly designed with built-in networking, making distributed computing system-independent.
The Package
The java.net package abstracts away low-level physical routing, allowing developers to focus purely on endpoints and data streams.
Network / IP
Data Link
Physical Wires

The Protocol Matrix: TCP vs. UDP

TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
	
Type: Connection-Oriented (guarantees delivery).	**Type:** Connectionless (fire and forget).
Speed: Slower (requires setup and acknowledgments).	**Speed:** Faster (no setup or guarantees).
Data Integrity: Reliable, error-checked, strictly ordered.	**Data Integrity:** Unreliable, independent packets, unordered.
Use Cases: Web applications (HTTP), File transfers (FTP), Emails.	**Use Cases:** Live streaming, Gaming, Video conferencing, Ping.
Java Classes: Socket, ServerSocket, URL.	**Java Classes:** DatagramPacket, DatagramSocket.

Locating the Target: IPs and Ports

IP Address (The Building)

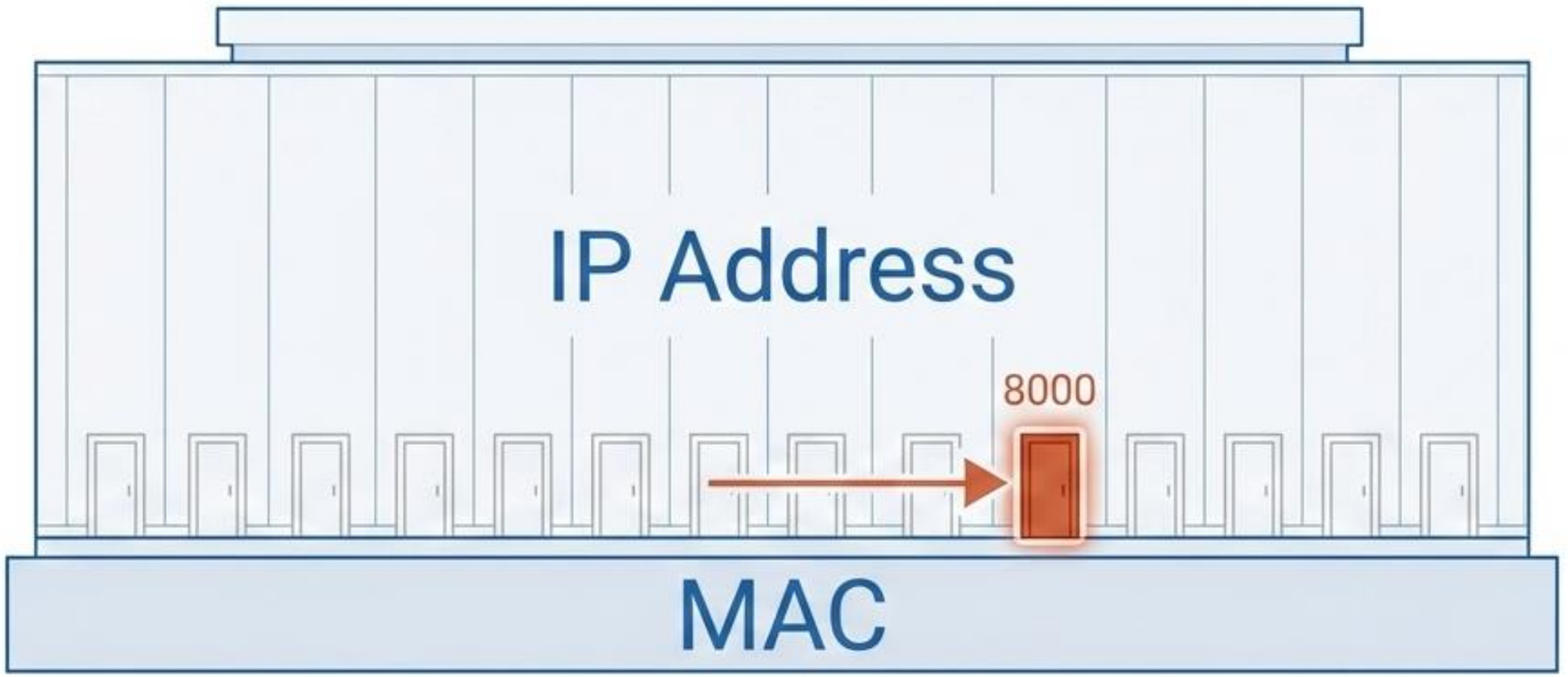
The 32-bit logical address identifying a specific device on the network (e.g., 192.168.0.1).

MAC Address (The Foundation)

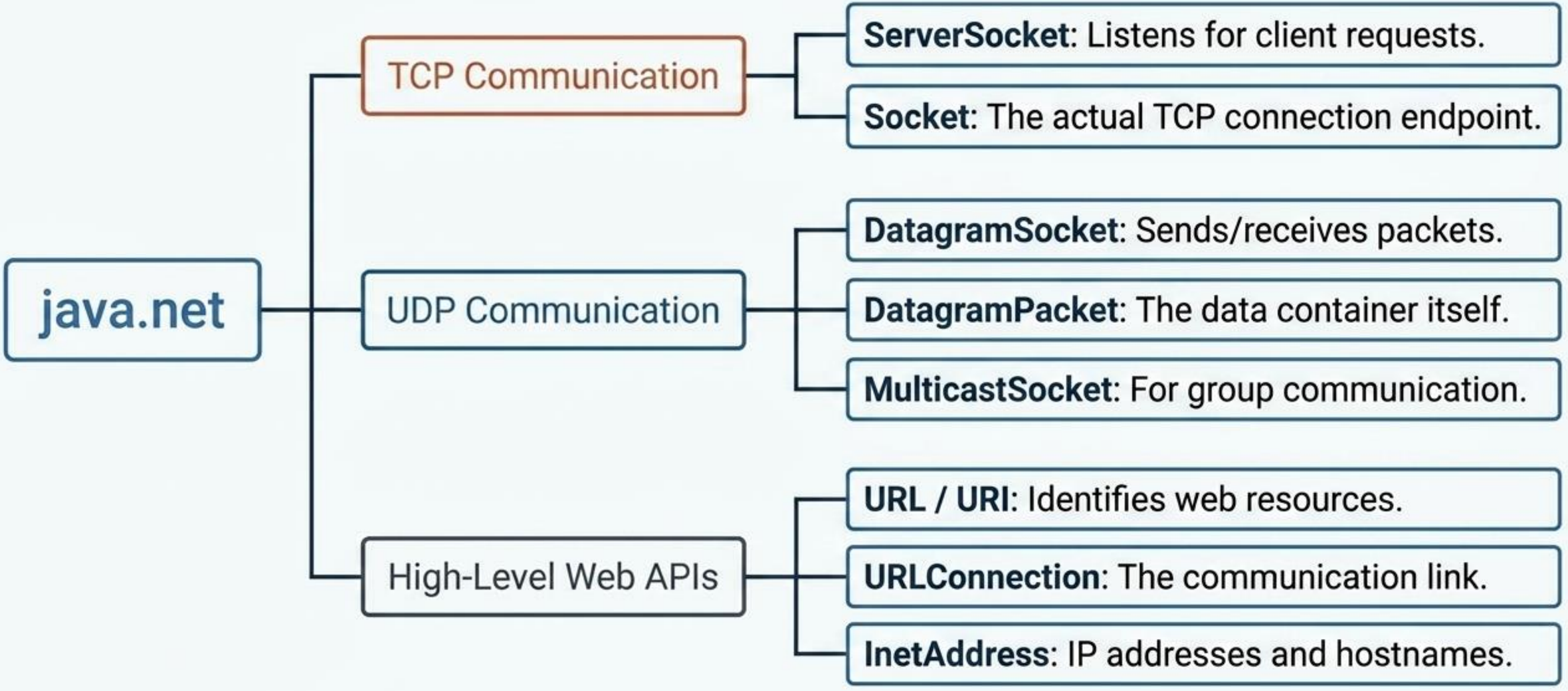
The permanent, hexadecimal hardware identifier of the network interface card.

Port Number (The Door)

A 16-bit number identifying the specific application receiving the data.
Range: 0 to 65,535.
Ports 0 to 1023 are reserved well-known ports.



The java.net Toolkit



The Socket Endpoint

Definition

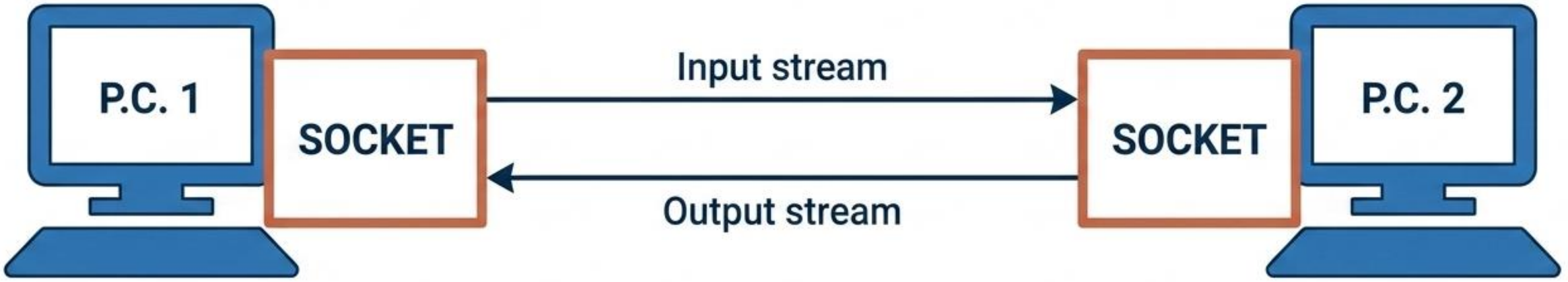
A socket is a single endpoint of a two-way communication link between two programs running on the network.

The Binding

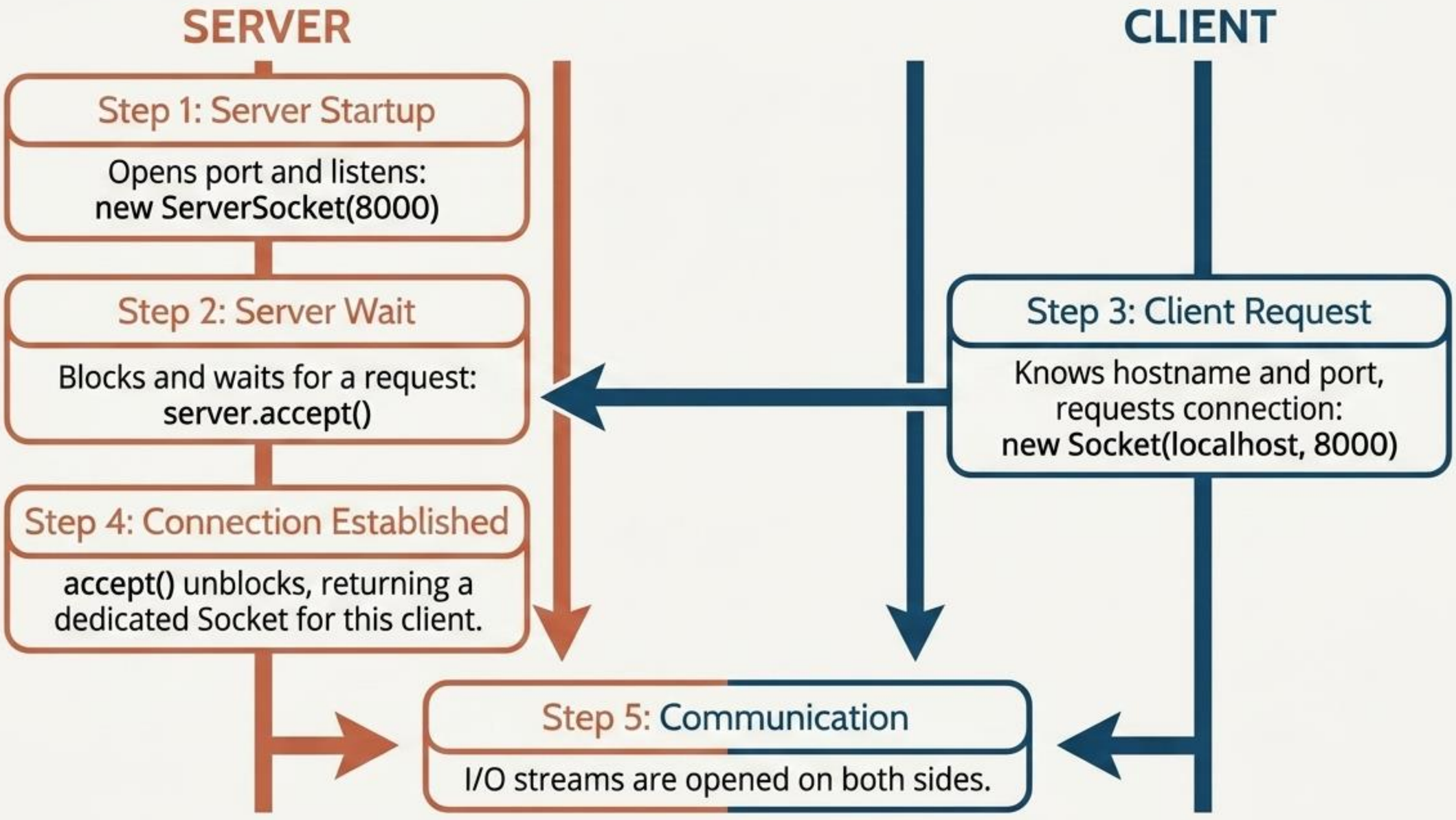
Every socket is bound to a specific port number so the network knows exactly which application the data is destined for.

The Rule

Each client socket is associated with exactly one remote host. Communicating with a new host requires creating a new socket.



The TCP Handshake Flowchart



The TCP Handshake Flowchart

1. The server runs on a specific computer and has a socket that is bound to a specific port number. ServerSocket server=new ServerSocket(8000);

2. The server just waits, listening to the socket for a client to make a connection request. Socket socket=server.accept();

3. The client knows the hostname of the machine on which the server is running and the port number on which the server is listening.

4.The client makes a connection request

Socket s=new Socket("127.0.0.1",8000);

3. The Server accepts the connection(using Socket socket=server.accept();)

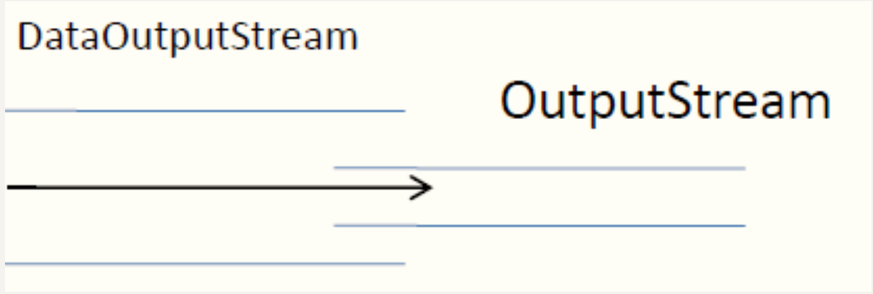
4. Upon acceptance, the server gets a new socket bound to the same local port and also has its remote endpoint set to the address and port of the client

DataInputStream / DataOutputStream

Note:

- The methods in class **InputStream & OutputStream** are not very useful
- Usually in networking we create objects from classes **DataInputStream / DataOutputStream**

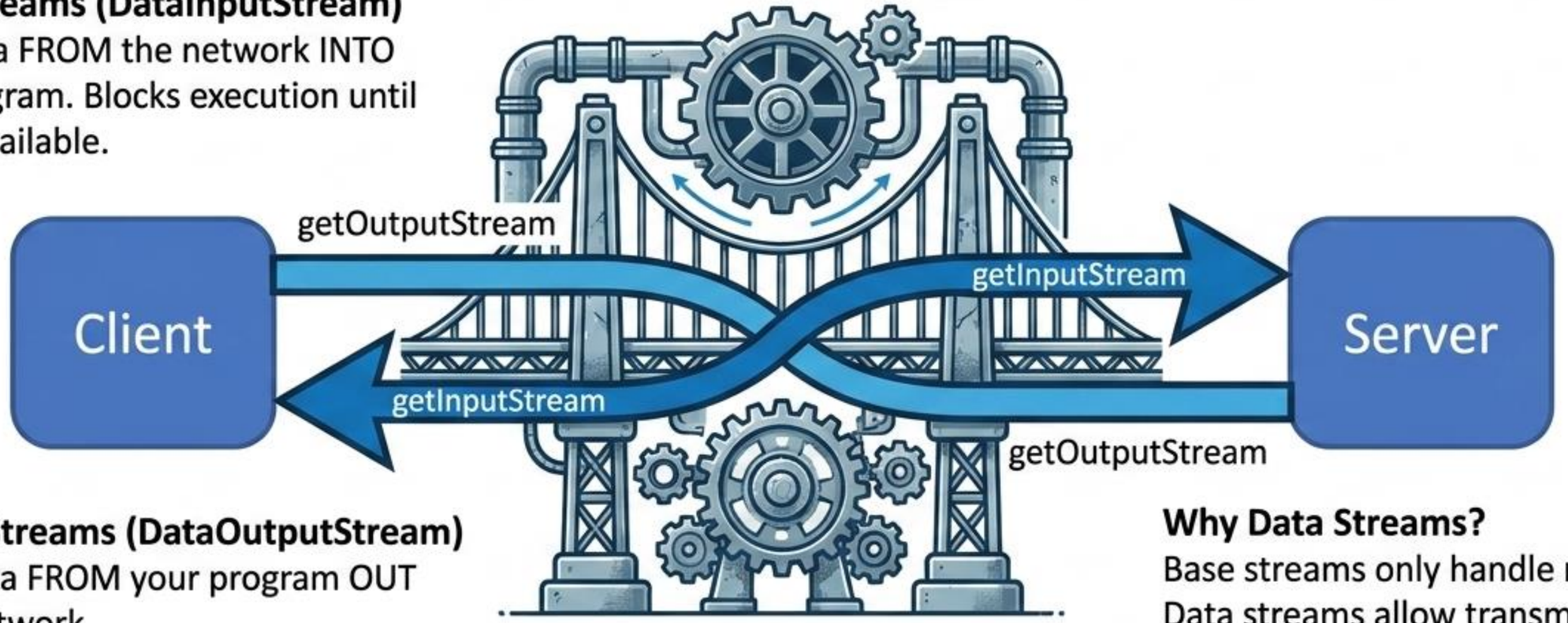
```
InputStream IS= s.getInputStream();
OutputStream OS=s.getOutputStream();
DataInputStreamreader =new DataInputStream(IS);
DataOutputStreamwriter=new DataOutputStream(OS);
```



The I/O Stream Bridge

The Principle
Streams are always relative to the current application.

Input Streams (DataInputStream)
Read data FROM the network INTO your program. Blocks execution until data is available.



Output Streams (DataOutputStream)
Write data FROM your program OUT to the network.

Why Data Streams?
Base streams only handle raw bytes. Data streams allow transmission of Java primitives (int, double, String).

Anatomy of a Server

1. Bind

```
ServerSocket ss = new ServerSocket(6000);
```

(Opens port 6000 and registers with the system)

2. Listen & Accept

```
Socket s = ss.accept();
```

(Blocks execution until a client connects, returning a dedicated socket s)

3. Open Streams & Read

```
DataInputStream dis = new DataInputStream(s.getInputStream());
String message = dis.readUTF();
```

(Creates an input stream and reads the client's data)

4. Terminate

```
s.close();
ss.close();
```

(Frees the port and terminates the connection)

Anatomy of a Client

1. Connect

```
Socket s = new Socket(localhost, 6000);
```

(Locates the server via IP/Hostname and requests entry at port 6000)

2. Open Streams

```
DataOutputStream dos = new DataOutputStream(s.getOutputStream());
```

(Creates a pipeline to push data to the server)

3. Write & Flush

```
dos.writeUTF("Message from client");
dos.flush();
```

(Forces the client buffer to empty, ensuring data is sent over the network)

4. Terminate

```
s.close();
```

(Closes the endpoint)

Anatomy of a Client

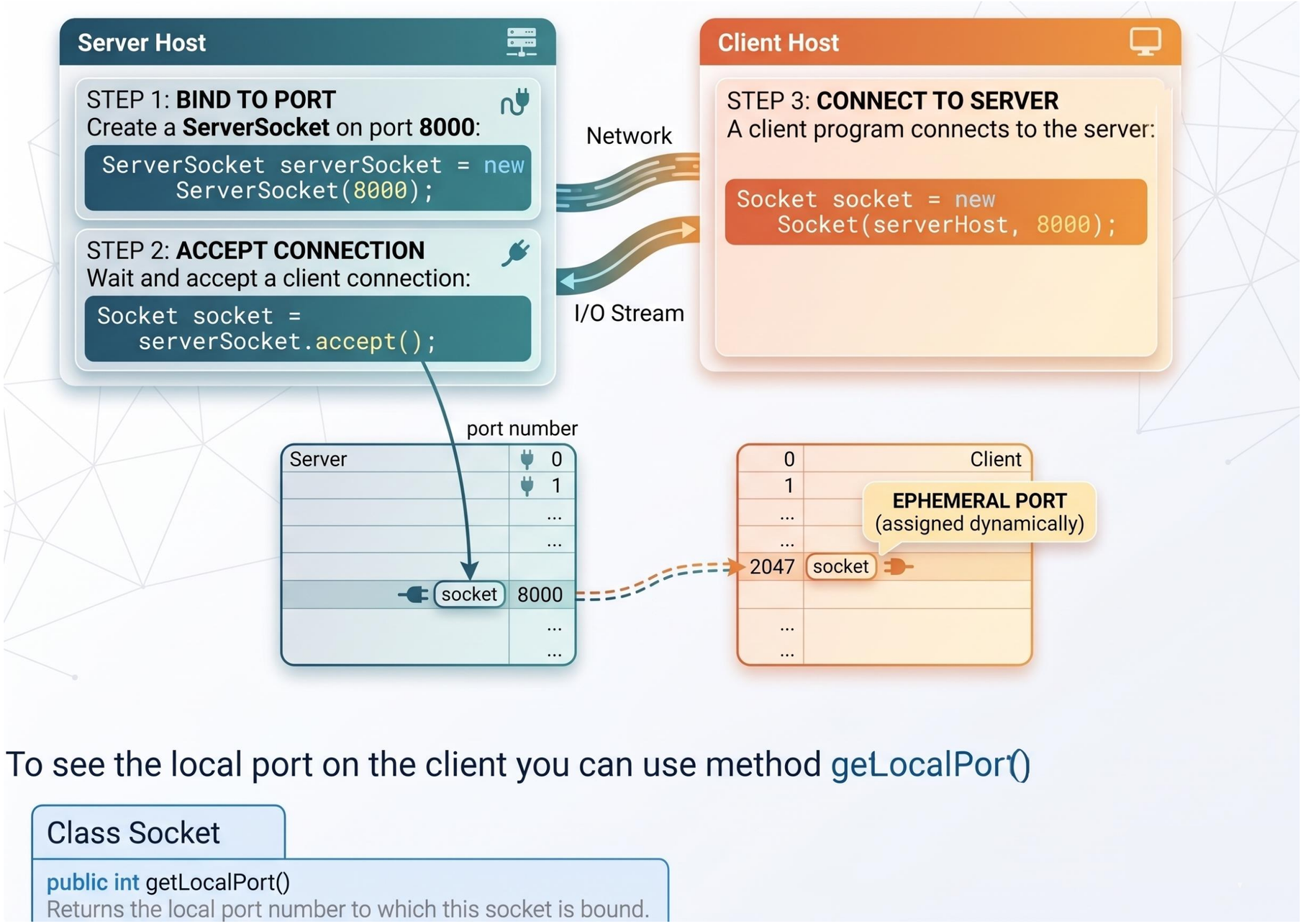
Alternatively, you can use the domain name to create a socket, as follows:

```
Socket socket= new Socket("liang.armstrong.edu", 8000);
```

- When you create a socket with a host name, the JVM asks the DNS to translate the host name into the IP address.

- A program can use the host name **localhost** or the IP address **127.0.0.1** to refer to the machine on which a client is running.

-The **Socket** constructor throws a **java.net.UnknownHostException** if the host cannot be found.



To see the local port on the client you can use method getLocalPort()

Class Socket

public int getLocalPort()
Returns the local port number to which this socket is bound.

Discovering Network Details

The **URL** Class: Parses string addresses.

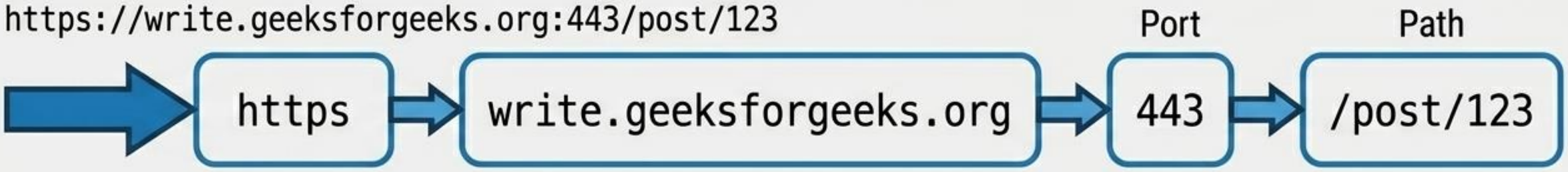


Diagram shows components broken down.

The **InetAddress** Class: The core IP modeling class.

- `InetAddress.getByName(simplylearn.com)`: Translates hostnames to IP addresses via DNS.
- `getHostAddress()`: Returns standard IP string (e.g., 192.168.1.1).
- `isLoopbackAddress()`: Checks if the address routes back to the same machine.
- `LocalHost`: The name of the current system, typically resolving to IP 127.0.0.1.

Discovering Network Details

Occasionally, you would like to know who is connecting to the server.

- You can use the ***InetAddressclass*** to find the client's ***host name*** and ***IP address***. The ***InetAddressclass*** models an IP address.

- You can use the statement shown below to create an instance of InetAddressfor the client on a socket.

```
InetAddressinetAddress= socket.getInetAddress();
```

Next, you can display the client's host name and IP address, as follows:

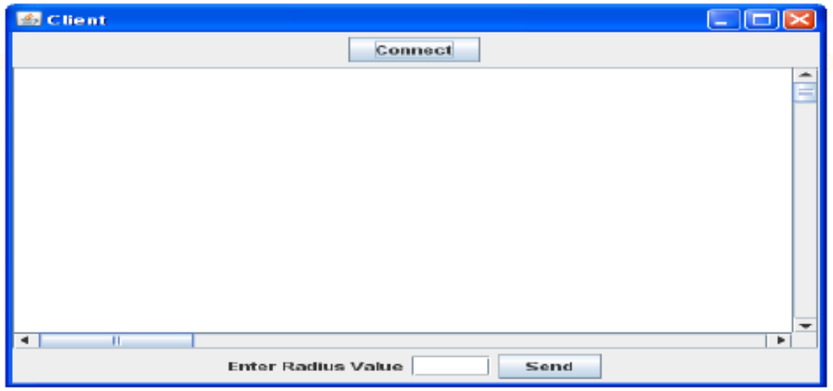
```
System.out.println("Client's host name is " + inetAddress.getHostName());
System.out.println("Client's IP Address is " + inetAddress.getHostAddress());
```

Client/ Server Example

- Problem: Write a client to send data to a server.
- The server receives the data, uses it to produce a result, and then sends the result back to the client.
- The client displays the result on the console. In this example, the data sent from the client is the radius of a circle, and the result produced by the server is the area of the circle.

Client Code

```
//////////////////
class  Client extends JFrame
{
JTextField t;
JTextArea  textArea;
DataInputStream reader;
DataOutputStream writer;
Client()
{
/////
textArea=new JTextArea(200,200);
JScrollPane js=new JScrollPane(textArea);
add(js,BorderLayout.CENTER);
add(getSouthPanel(),BorderLayout.SOUTH);
add(getNorthPanel(),BorderLayout.NORTH);}
```

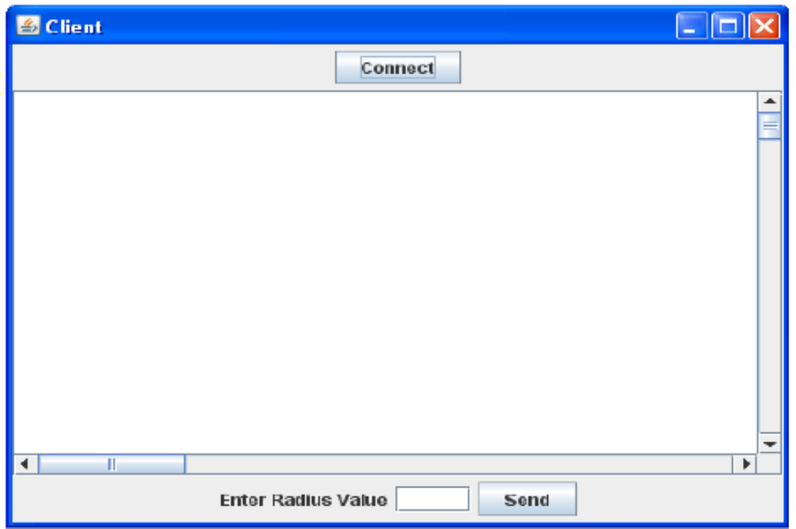



```
JPanel getNorthPanel(){
JPanel p=new JPanel();
final JButton connect=new JButton("Connect");
connect.addActionListener(newActionListener(){
public void actionPerformed (ActionEvent e){
try{
Socket s=new Socket("127.0.0.1",9000);
InputStream IS= s.getInputStream();
OutputStream  OS=s.getOutputStream();
reader=new DataInputStream(IS);
writer=new DataOutputStream(OS);
textArea.append("Connected to Server\n");
connect.setEnabled(false);}

catch (Exception exc){
JOptionPane.showMessageDialog(null,"can not
connect to server");}}});
p.add(connect);
return p;}
```

Client Code

```
JPanel getSouthPanel()
{
JPanel p=new JPanel();
JLabel l=new JLabel("Enter Radius Value");
t=new JTextField(4);
JButton send=new JButton("Send");
send.addActionListener(new ActionListener(){
public void actionPerformed (ActionEvent e)
{
communicate();
}
}
);
p.add(l);p.add(t);p.add(send);
return p;
}
```

```
void communicate(){
try{


writer.writeDouble(Double.parseDouble(t.getText()));
textArea.append("Value of radius
sent="+Double.parseDouble(t.getText())+"\n");
double area=reader.readDouble();
textArea.append("Value of Area="+area+"\n");
}
catch (Exception exc){
JOptionPane.showMessageDialog(null,"can not
send data");
}}
```



Server Code

```
//////
class  Server extends JFrame
{
///
Server()
{
////
t=new JTextArea(200,200);
////
try
{
server=new ServerSocket(9000);
t.append("Server running.....\n");
t.append("Waiting for connection.....\n");
}
catch (Exception exc){}
}
```

```
void ListenForConnection()
{
try
{
Socket socket=server.accept();
t.append("connection established\n");
InputStream IS= socket.getInputStream();
OutputStreamOS=socket.getOutputStream();
reader=new DataInputStream(IS);
writer=new DataOutputStream(OS);}
catch (IOException e){}}
```

```
public static void main(String [] arg)
{
Server server=new Server();
server.ListenForConnection();
server.communicate();
}
```

Server Code

```
void communicate(){
while (true)
try{
double radius=reader.readDouble();
t.append("Value of radius  recieved="+radius+"\n");
t.append("Calculating Area.....\n");
   t.append("Value of Area
="+calculateArea(radius)+"\n");
  writer.writeDouble(calculateArea(radius));}
catch(Exception e)
{}}
```

Scaling with Multithreading

The Bottleneck

A basic server processing a client's request blocks the accept() method. Other clients must wait in a queue.

The Solution

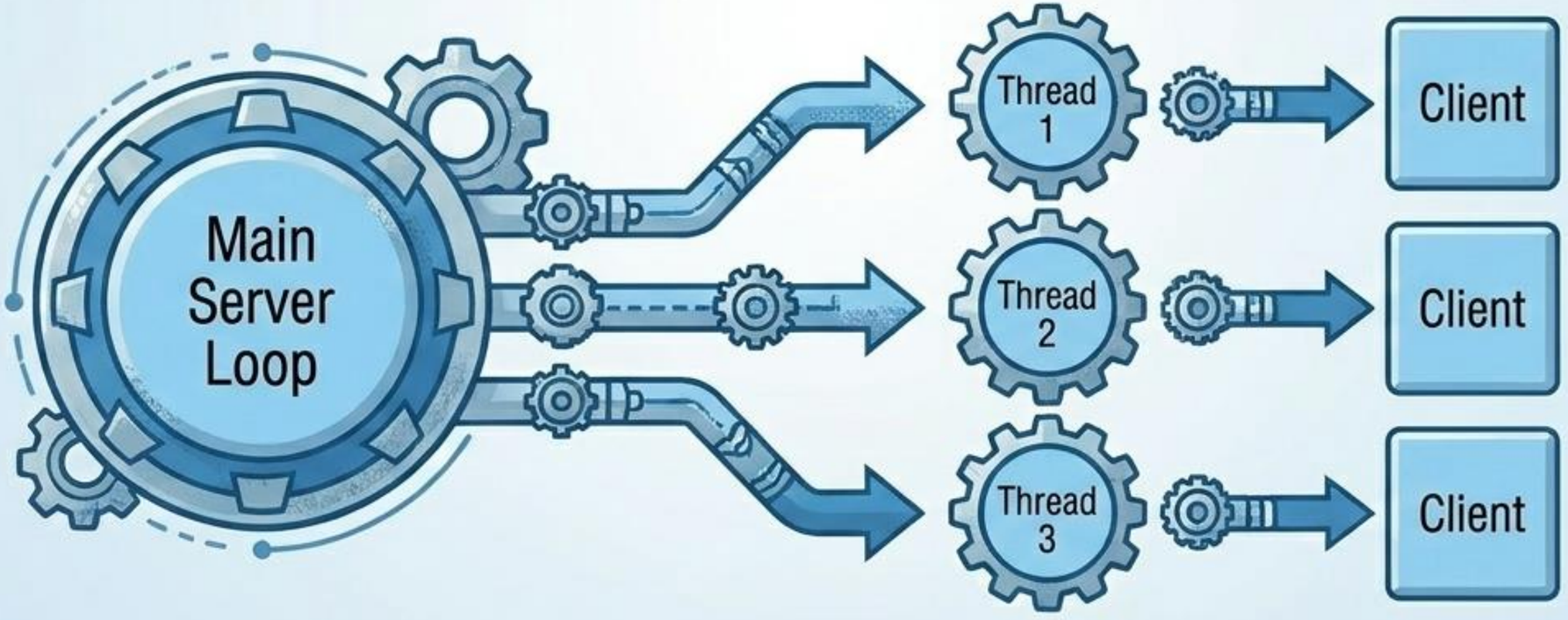
For every accepted connection, spawn a new Thread.

The Architecture

```
while(true) {
    Socket s = server.accept();
    Thread clientThread = new ThreadClass(s);
    clientThread.start();
}
```

Result

The server acts purely as a dispatcher, while worker threads handle the actual data I/O simultaneously.



Server

```
import java.io.*;        import java.net.*;        import java.util.concurrent.*;
public class MultiClientServer {
    private static final int PORT = 5000;

    public static void main(String[] args) {
        // Create a thread pool to manage up to 10 active clients
        ExecutorService threadPool = Executors.newFixedThreadPool(10);

        try (ServerSocket serverSocket = new ServerSocket(PORT)) {
            System.out.println("Server is listening on port " + PORT);

            while (true) {
                Socket socket = serverSocket.accept();
                System.out.println("New client connected: " + socket.getInetAddress());

                // Pass the client socket to a new handler task
                threadPool.execute(new ClientHandler(socket));
            }
        } catch (IOException e) {
            System.err.println("Server exception: " + e.getMessage());
        } finally {
            threadPool.shutdown();
        }    }   }
```

The server creates a ServerSocket and waits in a loop. When a client connects, it wraps that connection in a ClientHandler and hands it off to an ExecutorService.

Server

```
class ClientHandler implements Runnable {
    private Socket socket;

    public ClientHandler(Socket socket) {
        this.socket = socket;
    }
    @Override
    public void run() {
        try (
                InputStream input = socket.getInputStream();
                BufferedReader reader = new BufferedReader(new InputStreamReader(input));
                OutputStream output = socket.getOutputStream();
                PrintWriter writer = new PrintWriter(output, true)
        ) {      String text;
            while ((text = reader.readLine()) != null) {     System.out.println("Received: " + text);
                writer.println("Server echoed: " + text);

                if (text.equalsIgnoreCase("bye")) break;
            }
            socket.close();
        } catch (IOException e) {
            System.err.println("Client handler error: " + e.getMessage());
        }
    }     }
```

Client

```
import java.io.*;        import java.net.*;        import java.util.Scanner;

public class NetworkClient {
    public static void main(String[] args) {
        String hostname = "localhost";
        int port = 5000;

        try (Socket socket = new Socket(hostname, port)) {
            OutputStream output = socket.getOutputStream();
            PrintWriter writer = new PrintWriter(output, true);

            InputStream input = socket.getInputStream();
            BufferedReader reader = new BufferedReader(new InputStreamReader(input));

            Scanner scanner = new Scanner(System.in);
            String text;

            System.out.println("Connected to server. Type your message (or 'bye' to quit):");
```

Client

```
do {
        System.out.print("> ");
        text = scanner.nextLine();
        writer.println(text);

        String response = reader.readLine();
        System.out.println(response);

        } while (!text.equalsIgnoreCase("bye"));

        } catch (UnknownHostException e) {
        System.err.println("Server not found: " + e.getMessage());
        } catch (IOException e) {
        System.err.println("I/O error: " + e.getMessage());
        }
        }
        }
```

Handling Network Exceptions

java.net.BindException

Cause: Attempting to start a ServerSocket on a port that is already in use by another application.

java.net.ConnectException (Connection Refused)

Cause: Client attempts to connect, but the server is not running, or the client specified the wrong port number.

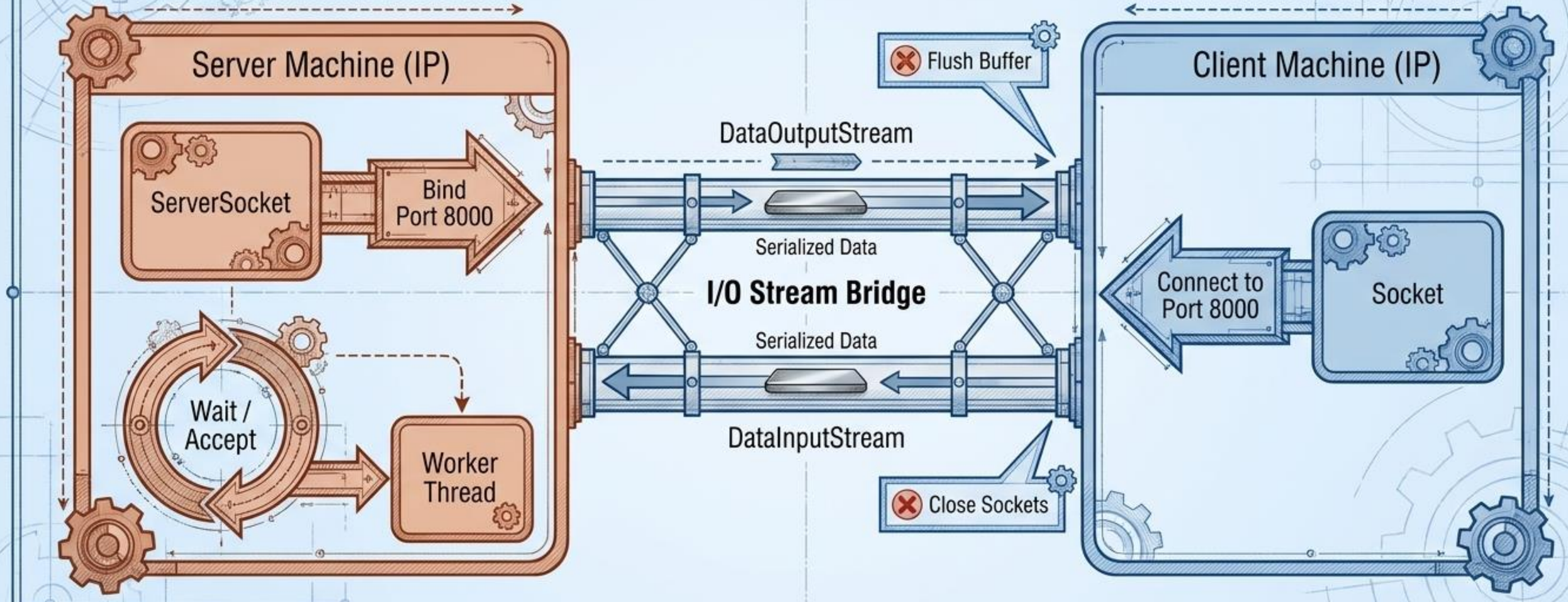
java.io.EOFException (End of File)

Cause: Server or client is actively trying to read from a stream, but the other end has unexpectedly closed the connection.

java.net.UnknownHostException

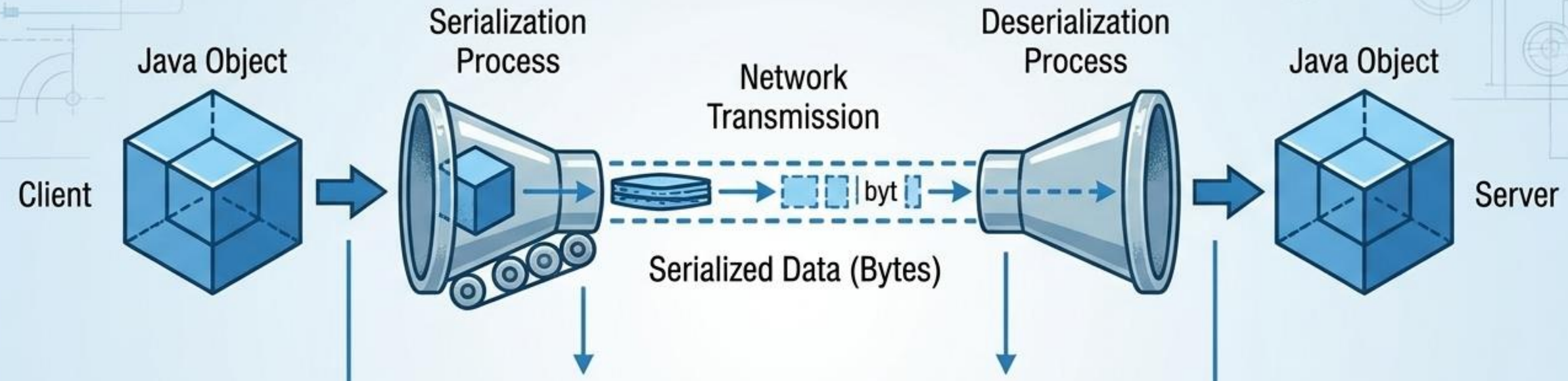
Cause: The JVM asked the DNS to translate a hostname, but no IP address could be found.

The Complete Connection Lifecycle



Network programming in Java translates the physical chaos of the internet into ordered, stream-based logic at the Application Layer.

Transmitting Java Objects



The Requirement

To send an entire object across a network, its class must implement the java.io.Serializable interface.

The Object Streams

ObjectOutputStream (socket.getOutputStream())

ObjectInputStream(socket .getInputStream())

The Mechanism

Client:
out.writeObject(studentObject);
Server:
Student s = (Student) in.readObject();

Use Case

Sending complex data structures, stateful configurations, or entire database records between client and server in a single network call.

Assignment

Use Serializable Interface and JavaFx to build a complete Client/Server Socket Communication

{Sending and Receiving a Text File}